

V3 - p2_report

March 19, 2026

Author: Yuze Fu

The goal of this project is to investigate the properties of standard finite difference methods for solving initial boundary value problems for linear hyperbolic PDEs. In particular, I am going to test and discuss the order of convergence and the stability for the following methods:

1. Lax-Friedrichs scheme with 1 ghost cell,
2. RK4CDS scheme with 1 ghost cell,
3. RK4CDS scheme with one-sided difference formula,
4. RK4C4 with 3 ghost cells.

1 Design

1.1 Choice of PDE

I use the following PDE for testing:

$$\begin{cases} u_t(x, t) + cu_x(x, t) = 0, & a \leq x \leq b, 0 \leq t \leq T, \\ u(x, 0) = f(x), & a \leq x \leq b, \\ u(a, t) = g(t), & 0 \leq t \leq T. \end{cases}$$

whose solution is $u(x, t) = f(x - ct)$ if f is smooth. In this project, I fix

$$u(x, t) = \exp(-(x - ct)^2) + \cos(3(x - ct))$$

with

$$c = 2, a = 0 \text{ and } b = 1$$

1.2 Set-up of Experiment

To test the order of convergence and stability, the main variable we care about is m which is the number of nodes between a and b . Thus, I will fix the other variables

$$CFL = 0.5 \text{ and } T = 1$$

We know that the theoretical stability condition is $CFL \leq 1$. $CFL = 0.5$ lies far enough from 1 and is less likely to cause an unexpected instability case. $T = 1$ is long enough for any potential instabilities to come out, while some large value like $T = 10$ may cause phase error.

In order to test the order of convergence and stability, we need both the ℓ_∞ -norm and ℓ_2 -norm of the errors which is the difference between $u_{estimated}$ and u_{exact} at time T . I will run the schemes

from $m = 20$ to $m = 10240$ and every iteration multiply 2 to m . Then, let's take the difference of the norms and Δ_x between two iterations.

Based on the definition, we have $E \approx C(\Delta x)^p$, and after applying log, we get $\log(E) \approx \log(C) + p \log(\Delta x)$. According to this, we can get the approximate order of convergence by

$$\frac{\log(diff_norm)}{\log(diff_dx)}$$

For example, the order assigned with the first iteration is 0 while for iteration $i > 0$, the approximate order is calculated by iteration i and $i + 1$.

By the values and trends of the order approximated by ℓ_∞ -norm, we can check them aligning with the theoretical order or not. If we found some potential issue (e.g. the approximate order decreases significantly), we can double check the order approximated by ℓ_2 -norm and two norms. Then, we may conclude the scheme becomes unstable when m increases.

2 Code

```
[20]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

def g(t, c, a):
    return np.exp(-np.pow(a - c * t, 2)) + np.cos(3 * (a - c * t))

def f(x):
    return np.exp(-np.pow(x, 2)) + np.cos(3 * x)

def run_lax_friedrichs(m, CFL=0.5, T=1, c=2, a=0, b=1):
    dx = (b - a) / (m - 1)
    xs = np.linspace(a, b, m)
    dt = CFL * dx / abs(c)
    nt = int(np.ceil(T / dt))
    dt = T / nt
    nu = c * dt / dx

    u_ghost = np.zeros(m + 1)
    u_ghost[:-1] = f(xs)
    for j in range(nt):
        u_ghost[0] = g(j * dt, c, a)
        u_ghost[-1] = 2 * u_ghost[-2] - u_ghost[-3]
        u_ghost[1:-1] = (1 / 2) * (u_ghost[2:] + u_ghost[:-2]) - (nu / 2) *
        ↪(u_ghost[2:] - u_ghost[:-2])

    u = u_ghost[:-1]
    u[0] = g(T, c, a)
```

```

u_exact = f(xs - c * T)
inf_error = np.max(np.abs(u - u_exact))
sq_error = np.sqrt(np.sum((u - u_exact)**2) * dx)
return xs, u, u_exact, dx, inf_error, sq_error

def get_L2(u, dx, c):
    l = np.zeros(np.shape(u)[0])
    l[1:-1] = -c * (u[2:] - u[:-2]) / (2 * dx)
    return l

def get_K2(u_0, t, c, a, dx):
    u = u_0.copy()
    u[0] = g(t, c, a)
    u[-1] = 2 * u[-2] - u[-3]
    return get_L2(u, dx, c)

def run_rk4cds_ghost(m, CFL=0.5, T=1, c=2, a=0, b=1):
    dx = (b - a) / (m - 1)
    xs = np.linspace(a, b, m)
    dt = CFL * dx / abs(c)
    nt = int(np.ceil(T / dt))
    dt = T / nt
    nu = c * dt / dx

    u_ghost = np.zeros(m + 1)
    u_ghost[:-1] = f(xs)
    for j in range(nt):
        k1 = get_K2(u_ghost, j * dt, c, a, dx)
        k2 = get_K2(u_ghost + 0.5 * dt * k1, (j + 0.5) * dt, c, a, dx)
        k3 = get_K2(u_ghost + 0.5 * dt * k2, (j + 0.5) * dt, c, a, dx)
        k4 = get_K2(u_ghost + dt * k3, (j + 1) * dt, c, a, dx)
        u_ghost = u_ghost + (dt / 6) * (k1 + 2 * k2 + 2 * k3 + k4)

    u = u_ghost[:-1]
    u[0] = g(T, c, a)
    u_exact = f(xs - c * T)
    inf_error = np.max(np.abs(u - u_exact))
    sq_error = np.sqrt(np.sum((u - u_exact)**2) * dx)
    return xs, u, u_exact, dx, inf_error, sq_error

def get_L_one_side(u, dx, c):
    l = np.zeros(np.shape(u)[0])
    l[1:-1] = -c * (u[2:] - u[:-2]) / (2 * dx)
    l[-1] = -c * (3 * u[-1] - 4 * u[-2] + u[-3]) / (2 * dx)
    return l

```

```

def get_K_one_side(u_0, t, c, a, dx):
    u = u_0.copy()
    u[0] = g(t, c, a)
    return get_L_one_side(u, dx, c)

def run_rk4cds_one_side(m, CFL=0.5, T=1, c=2, a=0, b=1):
    dx = (b - a) / (m - 1)
    xs = np.linspace(a, b, m)
    dt = CFL * dx / abs(c)
    nt = int(np.ceil(T / dt))
    dt = T / nt
    nu = c * dt / dx

    u = f(xs)
    for j in range(nt):
        k1 = get_K_one_side(u, j * dt, c, a, dx)
        k2 = get_K_one_side(u + 0.5 * dt * k1, (j + 0.5) * dt, c, a, dx)
        k3 = get_K_one_side(u + 0.5 * dt * k2, (j + 0.5) * dt, c, a, dx)
        k4 = get_K_one_side(u + dt * k3, (j + 1) * dt, c, a, dx)
        u = u + (dt / 6) * (k1 + 2 * k2 + 2 * k3 + k4)

    u[0] = g(T, c, a)
    u_exact = f(xs - c * T)
    inf_error = np.max(np.abs(u - u_exact))
    sq_error = np.sqrt(np.sum((u - u_exact)**2) * dx)
    return xs, u, u_exact, dx, inf_error, sq_error

def get_L4(u, dx, c):
    l = np.zeros(np.shape(u)[0])
    l[2:-2] = -c * (-u[4:] + 8 * u[3:-1] - 8 * u[1:-3] + u[:-4]) / (12 * dx)
    return l

def get_K4(u_0, t, c, a, dx):
    u = u_0.copy()
    u[1] = g(t, c, a)
    u[0] = 5 * u[1] - 10 * u[2] + 10 * u[3] - 5 * u[4] + u[5]
    u[-2] = 5 * u[-3] - 10 * u[-4] + 10 * u[-5] - 5 * u[-6] + u[-7]
    u[-1] = 15 * u[-3] - 40 * u[-4] + 45 * u[-5] - 24 * u[-6] + 5 * u[-7]
    return get_L4(u, dx, c)

def run_rk4c4_ghost(m, CFL=0.5, T=1.0, c=2, a=0, b=1):
    dx = (b - a) / (m - 1)
    xs = np.linspace(a, b, m)
    dt = CFL * dx / abs(c)
    nt = int(np.ceil(T / dt))

```

```

dt = T / nt
nu = c * dt / dx

u_ghost = np.zeros(m + 3)
u_ghost[1:-2] = f(xs)
for j in range(nt):
    k1 = get_K4(u_ghost, j * dt, c, a, dx)
    k2 = get_K4(u_ghost + 0.5 * dt * k1, (j + 0.5) * dt, c, a, dx)
    k3 = get_K4(u_ghost + 0.5 * dt * k2, (j + 0.5) * dt, c, a, dx)
    k4 = get_K4(u_ghost + dt * k3, (j + 1) * dt, c, a, dx)
    u_ghost = u_ghost + (dt / 6) * (k1 + 2 * k2 + 2 * k3 + k4)

u = u_ghost[1:-2]
u[0] = g(T, c, a)
u_exact = f(xs - c * T)
inf_error = np.max(np.abs(u - u_exact))
sq_error = np.sqrt(np.sum((u - u_exact)**2) * dx)
inf_norm = np.max(np.abs(u))
sq_norm = np.sqrt(np.sum(u**2) * dx)
return xs, u, u_exact, dx, inf_error, sq_error, inf_norm, sq_norm

```

3 Test

3.1 Lax-Friedrichs

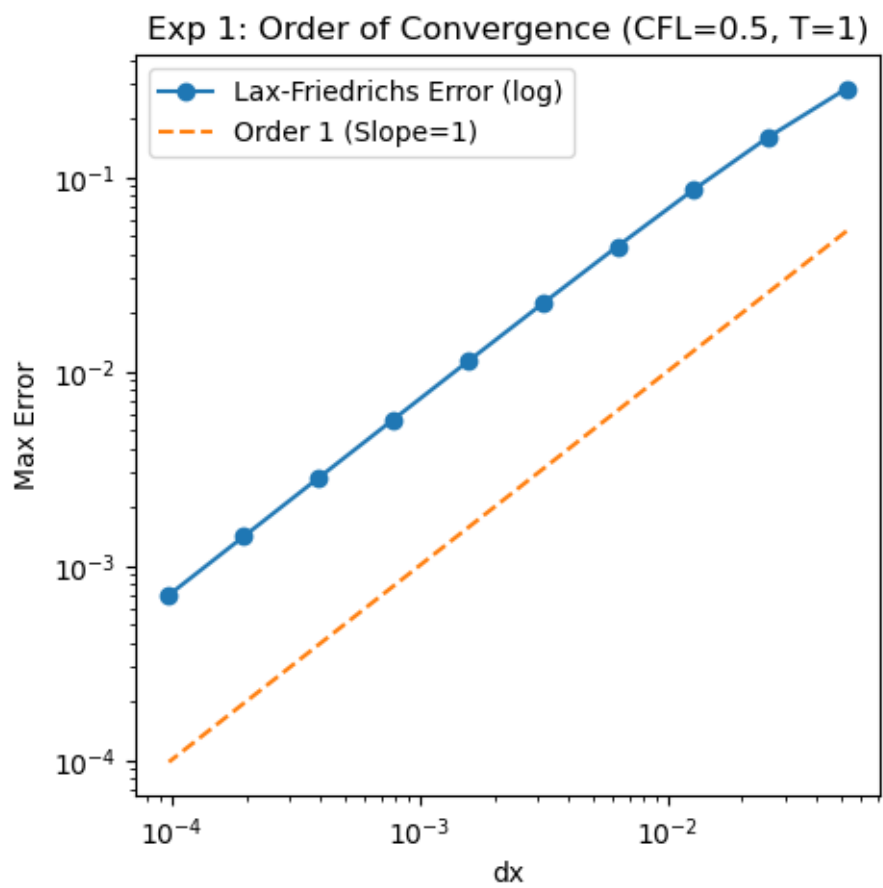
```
[21]: m = 20
dxs, inf_errors, sq_errors, ms = [], [], [], []
for i in range(10):
    _, _, u_exact, dx, inf_err, sq_err = run_lax_friedrichs(m)
    dxs.append(dx)
    inf_errors.append(inf_err)
    sq_errors.append(sq_err)
    ms.append(m)
    m *= 2

inf_orders, sq_orders = [0], [0]
inf_orders.extend(np.diff(np.log(inf_errors)) / np.diff(np.log(dxs)))
sq_orders.extend(np.diff(np.log(sq_errors)) / np.diff(np.log(dxs)))
df = pd.DataFrame([ms, inf_errors, inf_orders, sq_errors, sq_orders]).
    ↪transpose()
df.columns = ['m', 'linf-norm error', 'Order conv(linf)', 'l2-norm error',
    ↪'Order conv (l2)']
print(df)

fig, axs = plt.subplots(1, 1, figsize=(5, 5))
axs.loglog(dxs, inf_errors, 'o-', label='Lax-Friedrichs Error (log)')
axs.loglog(dxs, [d for d in dxs], '--', label='Order 1 (Slope=1)')
axs.set_title("Exp 1: Order of Convergence (CFL=0.5, T=1)")
axs.set_xlabel("dx")
axs.set_ylabel("Max Error")
axs.legend()
```

	m	linf-norm error	Order conv(linf)	l2-norm error	Order conv (l2)
0	20.0	0.284948	0.000000	0.165930	0.000000
1	40.0	0.161193	0.792225	0.086641	0.903586
2	80.0	0.085557	0.897336	0.043667	0.970666
3	160.0	0.044010	0.950405	0.021819	0.991938
4	320.0	0.022308	0.975832	0.010892	0.997842
5	640.0	0.011229	0.988097	0.005440	0.999415
6	1280.0	0.005633	0.994097	0.002718	0.999834
7	2560.0	0.002821	0.997061	0.001358	0.999949
8	5120.0	0.001412	0.998533	0.000679	0.999982
9	10240.0	0.000706	0.999268	0.000340	0.999993

```
[21]: <matplotlib.legend.Legend at 0x1554b7510>
```



3.2 RK4CDS (1 Ghost Cell)

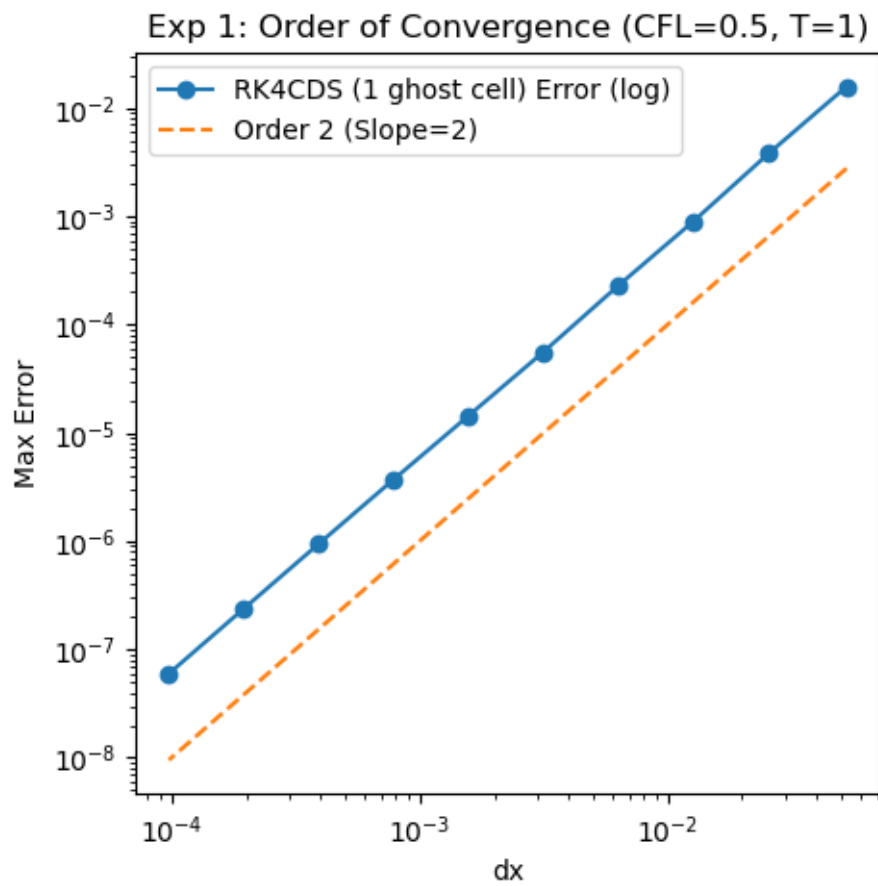
```
[22]: m = 20
dxs, inf_errors, sq_errors, ms = [], [], [], []
for i in range(10):
    _, _, u_exact, dx, inf_err, sq_err = run_rk4cds_ghost(m)
    dxs.append(dx)
    inf_errors.append(inf_err)
    sq_errors.append(sq_err)
    ms.append(m)
    m *= 2

inf_orders, sq_orders = [0], [0]
inf_orders.extend(np.diff(np.log(inf_errors)) / np.diff(np.log(dxs)))
sq_orders.extend(np.diff(np.log(sq_errors)) / np.diff(np.log(dxs)))
df = pd.DataFrame([ms, inf_errors, inf_orders, sq_errors, sq_orders]).
    ↪transpose()
df.columns = ['m', 'linf-norm error', 'Order conv(linf)', 'l2-norm error',
    ↪'Order conv (l2)']
print(df)

fig, axs = plt.subplots(1, 1, figsize=(5, 5))
axs.loglog(dxs, inf_errors, 'o-', label='RK4CDS (1 ghost cell) Error (log)')
axs.loglog(dxs, [d**2 for d in dxs], '--', label='Order 2 (Slope=2)')
axs.set_title("Exp 1: Order of Convergence (CFL=0.5, T=1)")
axs.set_xlabel("dx")
axs.set_ylabel("Max Error")
axs.legend()
```

	m	linf-norm error	Order conv(linf)	l2-norm error	Order conv (l2)
0	20.0	1.576624e-02	0.000000	8.370319e-03	0.000000
1	40.0	3.827985e-03	1.968415	1.985282e-03	2.000954
2	80.0	8.914668e-04	2.064392	4.841042e-04	1.999211
3	160.0	2.260969e-04	1.961387	1.195585e-04	1.999402
4	320.0	5.505860e-05	2.028712	2.971181e-05	1.999551
5	640.0	1.419653e-05	1.951021	7.406466e-06	1.999661
6	1280.0	3.672930e-06	1.948336	1.849049e-06	1.999745
7	2560.0	9.371402e-07	1.969484	4.619612e-07	1.999812
8	5120.0	2.374597e-07	1.980024	1.154560e-07	1.999865
9	10240.0	5.985513e-08	1.987856	2.886027e-08	1.999905

```
[22]: <matplotlib.legend.Legend at 0x15693ba50>
```

3.3 RK4CDS (One-sided Difference Formula)

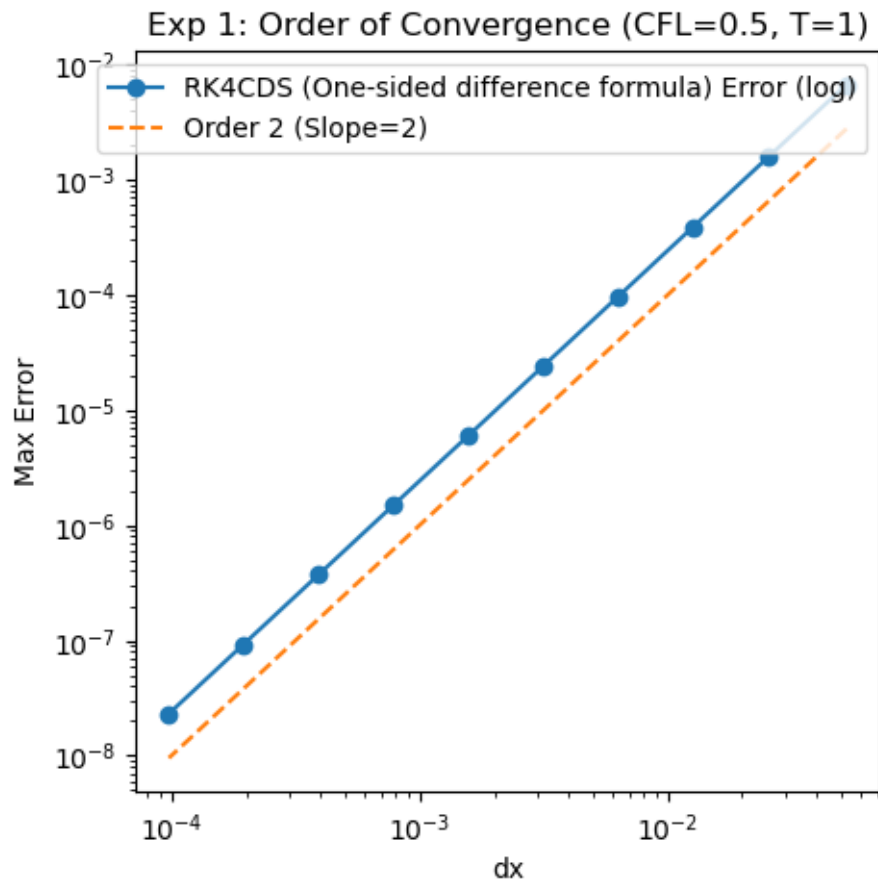
```
[23]: m = 20
dxs, inf_errors, sq_errors, ms = [], [], [], []
for i in range(10):
    _, _, u_exact, dx, inf_err, sq_err = run_rk4cds_one_side(m)
    dxs.append(dx)
    inf_errors.append(inf_err)
    sq_errors.append(sq_err)
    ms.append(m)
    m *= 2

inf_orders, sq_orders = [0], [0]
inf_orders.extend(np.diff(np.log(inf_errors)) / np.diff(np.log(dxs)))
sq_orders.extend(np.diff(np.log(sq_errors)) / np.diff(np.log(dxs)))
df = pd.DataFrame([ms, inf_errors, inf_orders, sq_errors, sq_orders]).
    ↪transpose()
df.columns = ['m', 'linf-norm error', 'Order conv(linf)', 'l2-norm error',
    ↪'Order conv (l2)']
print(df)

fig, axs = plt.subplots(1, 1, figsize=(5, 5))
axs.loglog(dxs, inf_errors, 'o-', label='RK4CDS (One-sided difference formula)
    ↪Error (log)')
axs.loglog(dxs, [d**2 for d in dxs], '--', label='Order 2 (Slope=2)')
axs.set_title("Exp 1: Order of Convergence (CFL=0.5, T=1)")
axs.set_xlabel("dx")
axs.set_ylabel("Max Error")
axs.legend()
```

	m	linf-norm error	Order conv(linf)	l2-norm error	Order conv (l2)
0	20.0	6.709541e-03	0.000000	4.043180e-03	0.000000
1	40.0	1.598323e-03	1.994897	9.920169e-04	1.953835
2	80.0	3.888249e-04	2.002562	2.464663e-04	1.972719
3	160.0	9.596407e-05	2.000347	6.145848e-05	1.985633
4	320.0	2.384627e-05	1.999673	1.534650e-05	1.992676
5	640.0	5.942635e-06	2.000070	3.834456e-06	1.996306
6	1280.0	1.483315e-06	2.000021	9.583481e-07	1.998145
7	2560.0	3.705402e-07	1.999995	2.395541e-07	1.999071
8	5120.0	9.259913e-08	1.999996	5.988443e-08	1.999535
9	10240.0	2.314520e-08	2.000004	1.497059e-08	1.999768

[23]: <matplotlib.legend.Legend at 0x156b4f990>



3.4 RK4C4 (3 Ghost Cells)

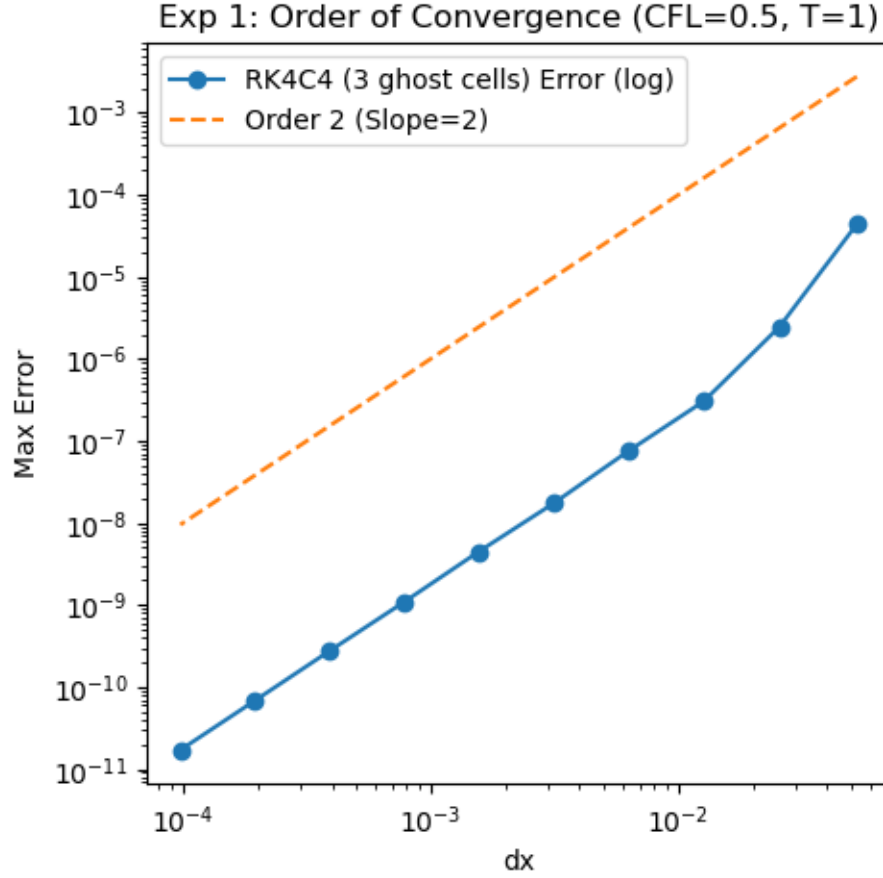
```
[26]: m = 20
dxs, inf_errors, inf_norms, sq_errors, sq_norms, ms = [], [], [], [], [], []
for i in range(10):
    _, _, u_exact, dx, inf_err, sq_err, inf_norm, sq_norm = run_rk4c4_ghost(m)
    dxs.append(dx)
    inf_errors.append(inf_err)
    sq_errors.append(sq_err)
    inf_norms.append(inf_norm)
    sq_norms.append(sq_norm)
    ms.append(m)
    m *= 2

inf_orders, sq_orders = [0], [0]
inf_orders.extend(np.diff(np.log(inf_errors)) / np.diff(np.log(dxs)))
sq_orders.extend(np.diff(np.log(sq_errors)) / np.diff(np.log(dxs)))
df = pd.DataFrame([ms, inf_errors, inf_orders, sq_errors, sq_orders]).
    ↪transpose()
df.columns = ['m', 'linf-norm error', 'Order conv(linf)', 'l2-norm error',
    ↪'Order conv (l2)']
print(df)

fig, axs = plt.subplots(1, 1, figsize=(5, 5))
axs.loglog(dxs, inf_errors, 'o-', label='RK4C4 (3 ghost cells) Error (log)')
axs.loglog(dxs, [d**2 for d in dxs], '--', label='Order 2 (Slope=2)')
axs.set_title("Exp 1: Order of Convergence (CFL=0.5, T=1)")
axs.set_xlabel("dx")
axs.set_ylabel("Max Error")
axs.legend()
```

	m	linf-norm error	Order conv(linf)	l2-norm error	Order conv (l2)
0	20.0	4.393565e-05	0.000000	2.760840e-05	0.000000
1	40.0	2.464209e-06	4.006069	1.278032e-06	4.272982
2	80.0	3.037916e-07	2.965470	9.764204e-08	3.643319
3	160.0	7.524419e-08	1.995268	1.098816e-08	3.123125
4	320.0	1.732210e-08	2.109411	1.440162e-09	2.918427
5	640.0	4.472530e-09	1.949047	2.114197e-10	2.761809
6	1280.0	1.088430e-09	2.036545	3.334751e-11	2.661456
7	2560.0	2.751709e-10	1.982730	5.894395e-12	2.498754
8	5120.0	6.835577e-11	2.008627	1.034050e-12	2.510330
9	10240.0	1.708889e-11	1.999723	1.834375e-13	2.494594

[26]: <matplotlib.legend.Legend at 0x156bbca90>



4 Summary

Method	Theoretical conv order	Actual conv order	Stable vs. Unstable
Lax-Friedrichs Scheme (ghost cells)	1	1	Stable
RK4CDS scheme (ghost cells)	2	2	Stable
RK4CDS scheme (one-sided difference formula)	2	2	Stable
RK4C4 scheme (ghost cells)	4	2	Unstable

(Note: Actual conv order is the number that the approximate order ends around when I increase m)

The first three methods are stable because all of their approximate conv orders are around their corresponding theoretical conv orders and don't have significant changes when we increase m .

We conclude RK4C4 with ghost cells is unstable since the approximate orders ends around 2 instead of 4 when I increase m . By observation, the approximate order by l_2 -norm goes down slower than the one by l_∞ -norm. Since l_2 -norm takes count of the whole u while l_∞ -norm only cares about the largest error, the observation can imply that when the errors of most points remain the same speed to decrease, the others start to decrease slower.

By educated guess, this phenomenon is caused by the extrapolation of our ghost cells which are the few special points starting to decrease slower than others and letting the approximate order by l_∞ -norm goes down. Though the extrapolation is 4-th order theoretically, it creates more discrepancy than we supposed. When the discrepancy of the ghost cell becomes large enough, the interior points will also be affected and cause the approxiamate order by l_2 -norm also goes down.